

CineSeat Backend

Toplantı Sunum Akışı

Visual Studio Üzerinden Canlı Kod Sunumu

Direktöre "ne yaptık" sorusuna Visual Studio'yu açıp adım adım cevap vermek için hazırlanmış bir akış. Her adımda **hangi klasörü/dosyayı açacağınız, ne söyleyeceğiniz ve ne göstereceğiniz** yazıyor. Sonunda olası sorulara hazır cevaplar var.

Tahmini süre: 12-15 dakika · Backend build durumu: 0 hata, doğrulandı

Sunumdan Önce (5 dakika önceden yapın)

Toplantı sırasında beklenmedik bir gecikme yaşamamak için bunları önceden hazırlayın.

1) PostgreSQL'i erkenden başlatın — veritabanı motoru açılışta birkaç saniye "ısınma" süresi istiyor. Toplantıdan en az 1 dakika önce başlatırsanız, F5 anında sorunsuz çalışır.

2) Visual Studio'yu önceden açın — `backend/CineSeat.slnx` 'i açın, `CineSeat.WebAPI` 'nin başlangıç projesi olduğunu kontrol edin (Solution Explorer'da kalın yazıyla görünür).

3) Bir kez F5'e basıp test edin — tarayıcıda Swagger sayfası otomatik açılmalı (`http://localhost:5207/swagger`). Açıldıysa kapatmayın, hazır bekleyin.

Sunum Akışı — Adım Adım

1 Açılış 30 sn

SÖYLE: "Backend'i ASP.NET Core Web API ile, CQRS ve MediatR mimarisiyle yazdık. Veritabanı PostgreSQL. Kod 5 ayrı projeye bölünmüş — Onion (soğan) mimarisi denen, sektörde yaygın kullanılan bir yapı kullandık. Şimdi baştan sona gezelim."

2 Solution Explorer — genel mimari 2 dk

GÖSTER: Solution Explorer'da 5 klasörü sırayla işaret edin: `Core/CineSeat.Domain` , `Core/CineSeat.Application` , `Infrastructure/CineSeat.Infrastructure` , `Infrastructure/CineSeat.Persistence` , `Presentation/CineSeat.WebAPI` .

SÖYLE: "Her katmanın net bir görevi var. Domain en içte — veritabanı tablolarımızın karşılığı olan sınıflar (Entity) burada, hiçbir şeye bağımlı değil. Application, iş kurallarımızın yaşadığı yer — az sonra detaylı göstereceğim. Persistence, veritabanı bağlantısını yönetiyor. WebAPI ise dışarıya açılan kapımız — HTTP isteklerini karşılıyor."

Klasör	Görevi
<code>Core/CineSeat.Domain</code>	21 Entity (Movie, User, Reservation...) + Enum'lar
<code>Core/CineSeat.Application</code>	CQRS — 45 Command + 31 Query, iş kuralları
<code>Infrastructure/CineSeat.Infrastructure</code>	JWT üretimi, şifre hash'leme
<code>Infrastructure/CineSeat.Persistence</code>	Veritabanı bağlantısı, migration'lar, repository'ler
<code>Presentation/CineSeat.WebAPI</code>	18 Controller, Program.cs, güvenlik middleware

SÖYLE (kritik cümle): "Bu 5 proje birbirine rastgele bağlı değil — her birinin `.csproj` dosyasında hangi projeye referans verebileceği tanımlı. Biri kuralı yanlışlıkla çiğnemeye çalışırsa proje **derlenmez**. Yani mimari kural, bir döküman değil, derleyicinin kendisi tarafından zorlanıyor."

3 Domain katmanı — veritabanının kod karşılığı 1 dk

GÖSTER: `Core/CineSeat.Domain/Entities/Movie.cs` 'yi açın.

SÖYLE: "Bu bir Entity — veritabanındaki 'movies' tablosunun C# karşılığı. 21 tane entity'miz var: Film, Kullanıcı, Rezervasyon, Bilet, Koltuk, Sinema, Salon, Kampanya gibi. Hepsi ortak bir `BaseEntity` 'den tüüyor — yani hepsinde otomatik `Id` , `CreatedAt` , `UpdatedAt` alanları var, tekrar tekrar yazmıyoruz."

GÖSTER: `Core/CineSeat.Application/Features/Cities/Commands/CreateCity/` klasörünü açın — 3 dosya göreceksiniz.

SÖYLE: "CQRS, 'Command Query Responsibility Segregation' — okuma ve yazma işlemlerini kod olarak birbirinden ayırıyoruz. Bir **Command** veri değiştirir (ekleme, güncelleme, silme), bir **Query** sadece okur. Her özellik — mesela 'yeni şehir ekle' — 3 küçük dosyaya bölünmüş."

`CreateCityCommand.cs`

```
public class CreateCityCommand : IRequest<long>
{
    public string CityName { get; set; } = string.Empty;
}
```

SÖYLE: "Bu sadece 'ne isteniyor' bilgisini taşıyor — hiç iş mantığı yok."

`CreateCityCommandHandler.cs`

```
public async Task<long> Handle(CreateCityCommand request, CancellationToken ct)
{
    var city = new City { CityName = request.CityName };
    await _cityWriteRepository.AddAsync(city, ct);
    await _cityWriteRepository.SaveAsync(ct);
    return city.Id;
}
```

SÖYLE: "İşin gerçekten yapıldığı yer burası — Handler. Her komutun kendine ait, tek bir işi yapan bir Handler'ı var."

`CreateCityCommandValidator.cs`

```
RuleFor(x => x.CityName).NotEmpty().WithMessage("Şehir adı boş olamaz.");
```

SÖYLE: "Ve doğrulama kuralı ayrı bir dosyada — bu, Handler çalışmadan **otomatik** devreye giriyor, hiç elle çağırmıyoruz."

GÖSTER: Şimdi `Presentation/CineSeat.WebAPI/Controllers/CitiesController.cs` 'i açın, Create metodunu gösterin.

```
[HttpPost]
public async Task<IActionResult> Create([FromBody] CreateCityCommand command)
    => Ok(await _mediator.Send(command));
```

SÖYLE (özet cümle): "MediatR dediğimiz kütüphane, bu komutu alıp doğru Handler'ı otomatik buluyor ve çalıştırıyor. Controller'ımız tek satır — hiçbir veritabanı kodu, hiçbir iş kuralı içermiyor. Bu sayede kodumuz hem çok düzenli, hem test edilmesi kolay, hem de yeni bir özellik eklerken var olan hiçbir dosyaya dokunmuyoruz — sadece yeni bir klasör ekliyoruz."

GÖSTER (isteğe bağlı, zaman varsa): `Common/Behaviors/ValidationBehavior.cs` ve `Common/Behaviors/LoggingBehavior.cs` — "her istek otomatik önce loglanıyor, sonra doğrulanıyor, sonra Handler'a gidiyor" deyip pipeline mantığını 20 saniyede özetleyin.

5 DTO'lar — dışarıya ne döndürüyoruz 1 dk

GÖSTER: `Features/Cities/DTOs/CityDto.cs` veya `Features/Reservations/DTOs/ReservationDto.cs` .

SÖYLE: "DTO — Data Transfer Object. Veritabanı Entity'sini olduğu gibi dışarı asla döndürmüyoruz; sadece istemcinin ihtiyacı olan alanları içeren ayrı bir sınıf kullanıyoruz. Bu hem güvenlik açısından önemli — mesela şifre hash'i hiçbir zaman API yanıtına karışmıyor — hem de API sözleşmemizi veritabanı yapımızdan bağımsız tutuyor."

6 Repository Pattern — veritabanına nasıl erişiyoruz 1.5 dk

GÖSTER: Önce `Core/CineSeat.Application/Repositories/ICityReadRepository.cs` (arayüz), sonra `Infrastructure/CineSeat.Persistence/Repositories/City/CityReadRepository.cs` (gerçekleştirim).

SÖYLE: "Application katmanı veritabanı teknolojisini (Entity Framework) hiç tanımıyor — sadece bir 'arayüz' biliyor: 'bana bir kayıt getir' diyor, nasıl getirileceğini bilmiyor. Gerçek veritabanı kodu Persistence projesinde. Bu sayede yarın veritabanını değiştiresek bile — PostgreSQL'den başka bir şeye geçsek — iş mantığı kodumuza **hiç dokunmayız**, sadece Persistence projesini değiştiririz."

SÖYLE (repository dosyaları neden kısa): "40 tane repository dosyamız var ama çoğu 5 satır — çünkü ortak mantık (listeleme, id ile arama, ekleme, silme) tek bir 'generic' temel sınıfta yazılı, her biri onu miras alıyor. Aynı kodu 40 kere kopyalamıyoruz."

7 Kimlik doğrulama (Auth) ve güvenlik 2 dk

GÖSTER: `Infrastructure/CineSeat.Infrastructure/Security/JwtTokenService.cs` ve `Pbkdf2PasswordHasher.cs` .

SÖYLE: "Kullanıcı giriş yaptığında ona JWT dediğimiz, imzalı bir 'kimlik kartı' (token) veriyoruz. Şifreler veritabanında asla düz metin olarak durmuyor — PBKDF2 denen, 100 bin kez tekrarlanan bir algoritmayla şifreleniyor, her kullanıcı için farklı bir 'tuz' (salt) değeriyle. Yani veritabanımız çalınsa bile şifreler çözülemez."

GÖSTER: Herhangi bir controller'da `[Authorize(Roles = RoleNames.Admin)]` satırını gösterin (örn. CitiesController'ın Create metodu).

SÖYLE: "Rol tabanlı yetkilendirme kullandık — Admin ve User rolleri var. Yazma işlemleri (ekleme/silme) sadece Admin'e açık, okuma işlemleri (film listesi, koltuk haritası gibi) herkese açık."

Güvenlik detayı (sorulursa): Bir kullanıcının id tahmin ederek başkasının rezervasyonunu görmesini engelledik (IDOR koruması) — her rezervasyon/bilet sorgusunda "bu kayıt gerçekten sana mı ait" kontrolü var, admin hariç herkes sadece kendi verisini görebiliyor.

8

En karmaşık özellik — Rezervasyon akışı

1.5 dk

GÖSTER:

`Features/Reservations/Commands/CreateReservation/CreateReservationCommandHandler.cs` dosyasını açın, kaydırıp genel uzunluğunu gösterin (en büyük Handler'ımız).

SÖYLE: "Rezervasyon oluşturma en çok adımlı iş kuralımız: seans ve koltuk var mı kontrolü, koltuğun daha önce rezerve edilip edilmediği, başka biri tarafından kilitli olup olmadığı, bilet fiyatı hesabı, varsa kampanya indirimi hesaplanması — hepsi **tek bir veritabanı işleminde (transaction)** kaydediliyor. Yani ya hepsi başarılı olur ya da hiçbiri — yarım kalmış bir rezervasyon asla oluşmaz."

SÖYLE (varsa zaman, koltuk kilidi anekdotu): "İki kişi aynı anda aynı koltuğu seçerse ne olur diye de düşündük — veritabanı seviyesinde bir kısıt koyduk, aynı koltuk aynı seans için asla iki kere kilitlenemez."

9

CANLI DEMO — Swagger üzerinden çalıştırma

3 dk

GÖSTER: Tarayıcıda açık olan Swagger sayfasına geçin (`/swagger`).

SÖYLE: "Bu sayfa, tüm API'mizin otomatik oluşturulmuş canlı dökümanı — kod değişince otomatik güncelleniyor, elle yazmıyoruz."

1. `POST /api/auth/login` 'u açın → `{"usernameOrEmail":"admin@cineseat.com","password":"Admin123!"}` ile "Execute" → dönen `token` 'i kopyalayın.
2. Sağ üstteki **Authorize** butonuna basıp token'ı yapıştırın.
3. `GET /api/movies` 'i çalıştırın → sayfalanmış film listesi dönsün.
4. `GET /api/cities` 'i çalıştırın → şehir listesi.

SÖYLE: "Gördüğünüz gibi backend gerçekten çalışıyor, gerçek bir PostgreSQL veritabanına bağlı ve gerçek veriler dönüyor."

10

Kapanış — rakamlarla özet

30 sn

5

PROJE

21

ENTITY

18

CONTROLLER

45

COMMAND

31

QUERY

0

BUILD HATASI

SÖYLE: "Özetle: kimlik doğrulama, yetkilendirme, film/sinema/salon yönetimi, koltuk rezervasyonu, kampanya indirimleri, kullanıcı yorumları/favorileri — hepsi çalışır durumda, test edildi ve şu an sıfır derleme hatasıyla ayakta. Sorularınızı alabilirim."

Olası Sorular ve Hazır Cevaplar

Direktör araya girip soru sorarsa, hazır cevap için buraya bakın.

S: Neden bu kadar çok proje/klasöre böldünüz, tek proje olamaz mıydı?

C: Olabilirdi ama katmanları ayrı proje yapmak, kuralları derleyiciye zorlatıyor. Biri yanlışlıkla "kısayol" yapıp veritabanı kodunu iş mantığının içine sızdırmaya çalışsa, proje derlenmez. Küçük bir ekipte bile bu disiplin uzun vadede kodun bozulmasını engelliyor.

S: Test yazdınız mı?

C: Otomatik (unit/integration) test dosyası şu an yok; ama her özellik gerçek HTTP istekleriyle uçtan uca elle test edildi — kayıt, giriş, yetkilendirme, rezervasyon, kampanya indirimi, güvenlik senaryoları (yetkisiz erişim engelleri) dahil.

S: Güvenlik açısından neler yaptınız?

C: Şifreler PBKDF2 ile 100.000 iterasyonla hash'leniyor, her kullanıcı için ayrı rastgele "tuz" ile. JWT token'lar imzalı. Rol tabanlı yetkilendirme var (Admin/User). Kullanıcıların birbirinin verisine (rezervasyon, bilet) id tahmin ederek erişmesi engellendi — bulduğumuz güvenlik açıklarını (IDOR) kapattık.

S: Bu mimarinin ismi ne, nereden geliyor?

C: Onion (Soğan) / Clean Architecture — bağımlılıkların hep "içeri" (Domain'e doğru) aktığı, sektörde yaygın kabul görmüş bir yaklaşım. CQRS + MediatR ise iş mantığını Command/Query olarak ayıran, büyük .NET projelerinde standart sayılan bir tasarım deseni.

S: Ne kadar sürdü?

C: [Ekip kendi gerçek zaman çizelgenizi buraya ekleyin — plan dokümanında fazlara bölünmüş haldeydi: altyapı, katalog, sosyal özellikler, rezervasyon, güvenlik.]

S: Eksik/yapılacak bir şey var mı?

C: Ana akışların hepsi tamam. Küçük iyileştirme fırsatları not edildi (ör. rol/izin yönetimi için ayrı bir admin ekranı, otomatik test paketi) — bunlar sonraki aşamada planlanabilir, mevcut işlevi etkilemiyor.